

[_3D.ShowUserDefinedAttributes \[SimTalk\]](#)

Shows (`true`) or hides (`false`) the *user-defined attributes* of the objects in the *Frame* designated by `<Path>`.

Type

Attribute

Syntax

```
<Path>._3D.ShowUserDefinedAttributes:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyFrame._3D.ShowUserDefinedAttributes := false
```

See also

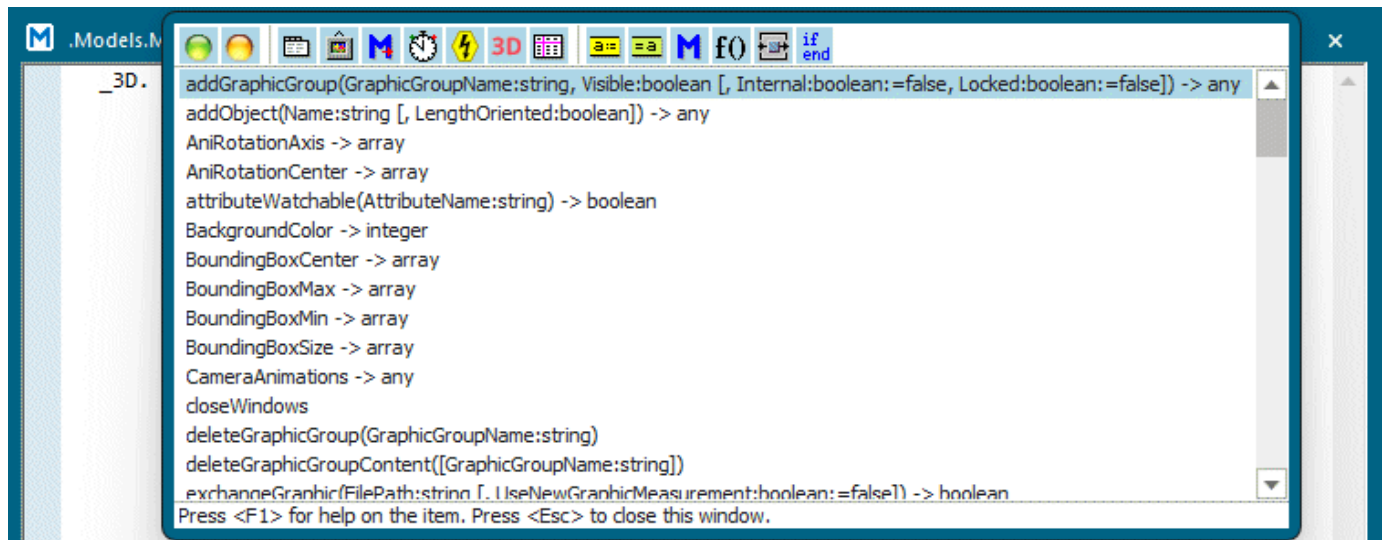
[View Ribbon Tab > Show User-defined Attributes](#)

Accessing Objects in 3D

[__Accessing Objects in 3D](#)

SimTalk provides the *attributes* and *methods* listed in the table of contents to the left for accessing objects *3D*.

You can view and access the **attributes and methods** for accessing objects in *3D* by clicking **Auto Complete** on the **Edit** ribbon tab of the *Method Editor*.



See also

[Auto Complete](#)

[_3D.addLengthOrientation \[SimTalk\]](#)

Makes a *length-oriented object* out of the object designated by <Path>.

Remarks

This does not apply to existing length-oriented objects, moving objects (*MUs*), *Connectors*, and *Interfaces*.

Type

Method

Syntax

```
<Path>._3D.addLengthOrientation
```

Example

```
MyStation._3D.addLengthOrientation
```

See also

[_3D.removeLengthOrientation \[SimTalk\]](#)

[_3D.InternalClassType \[SimTalk\]](#)

[_3D.addObject \[SimTalk\]](#)

Creates an animatable object for the object designated by <Path>.

Type

Method

Syntax

```
<Path>._3D.addObject(Name:string[, LengthOriented:boolean]) → any
```

Parameters

You can specify the following parameters:

- The parameter *Name* of data type *string* designates the name of the animatable object.
Specify a name, which corresponds to the name of an animatable object in the origin of the object, in which the new animatable object is going to be created to create an object that inherits from the animatable object in the origin of the respective object. This way you can restore a deleted animatable object, which inherits from another object, in a way that it inherits again from the original object.
- The optional parameter *LengthOriented* of data type *boolean* sets if the animatable object is to be length-oriented (*true*) or not (*false*).

If you specify this parameter and the object would be created with an origin, the newly created object will only have an origin if this origin is length-oriented as well if you set it to *true*.

This is not the case if you specify *false*.

Specify this parameter to make the new object length-oriented if it has an origin and if this origin is length-oriented as well.

Return Value

The return value has the data type *any*.

It is the newly created object.

Examples

```
// adds an animatable object to the Station  
var MyAnimatableObject := Station._3D.addObject("MyAnimatableObject")  
  
// deletes the animatable object from the Station
```

```
MyAnimatableObject.delete  
Station._3D.getObject("MyAnimatableObject").delete
```

```
// adds an animatable object to the Conveyor  
// and makes it length-oriented  
var MyAnimatableObject := Conveyor._3D.addObject("MyAnimatableObject", true)
```

See also

[Make Animatable Object](#)

[makeAnimatableObject \[SimTalk\]](#)

[Add Animatable Object](#)

[_3D.getObject \[SimTalk\]](#)

[delete \[SimTalk\] - animatable object](#)

[_3D.Exists \[SimTalk\]](#)

Checks if the object designated by <Path> exists in 3D (true) or not (false).

Remarks

If the object exists or not matters if the existence of a 3D object depends on its context. A *Connector* in 3D only exists if its predecessor as well as its successor exist in 3D.

Type

Read-only attribute

Syntax

```
<Path>._3D.Exists → boolean
```

Return Value

The return value has the data type *boolean*.

Example

```
if obj._3D.Exists  
...  
end
```

See also

[_3D.existsObject \[SimTalk\]](#)

[_3D.existsObject \[SimTalk\]](#)

Checks if the animatable object designated by <Path> exists in 3D (true) or not (false).

Type

Method

Syntax

```
<Path>._3D.existsObject(Name:string) → boolean  
<Path>._3D.existsObject(Index:integer) → boolean
```

Parameter

The parameter *Name* of data type *string* designates the name of the object.

Or

The parameter *Index* of data type *integer* designates the n-th internal animatable object.

Return Value

The return value has the data type *boolean*.

Example

```
print basis._3D.existsObject("MyObject")
```

See also

[_3D.Exists \[SimTalk\]](#)

[existsObject \[SimTalk\]](#)

[_3D.ExistsWithAnimation \[SimTalk\]](#)

Returns if the object designated by <Path> exists in 3D or not and if the respective animation function is activated (true) or not (false).

Remarks

For *MUs* the animation function is the **MU Animation**, for all other objects it is the **States Animation**.

_3D.ExistsWithAnimation applies to all *MUs* and simulation objects. It does not apply to the *folder*, *Connectors*, and *Interfaces*.

Type

Read-only attribute

Syntax

```
<Path>._3D.ExistsWithAnimation → boolean
```

Return Value

The return value has the data type *boolean*.

Example

```
print MyStation._3D.ExistsWithAnimation
```

See also

[_3D.Exists \[SimTalk\]](#)

[_3D.existsObject \[SimTalk\]](#)

[MUs and States \[Home ribbon\]](#)

[_3D.getAttribute \[SimTalk\]](#)

Returns the value of the specified attribute of the object designated by <Path>.

Remarks

`_3D.getAttribute` also applies to animatable objects.

Type

Method

Syntax

```
<Path>._3D.getAttribute(AttributeName:string[, byRef Inherited:boolean,  
byRef CanInherit:boolean]) → any
```

Parameters

You can specify the following parameters:

- The parameter *AttributeName* of data type *string* designates the value of the attribute.
- If a local variable of data type *boolean* is passed to the optional parameter *Inherited*, *Plant Simulation* assigns `true` to this variable if the attribute designated by string *inherits* its value, `false` if it does not inherit the value.

In case an attribute cannot be inherited, the optional parameter *Inherited* always returns `false`.

- If a local variable of data type *boolean* is passed to the optional parameter *CanInherit*, *Plant Simulation* assigns `true` to this variable if the value of the attribute designated by string can be inherited, `false` if it cannot be inherited.

Return Value

The return value has the data type *any*.

Examples

```
var inherited, canInherit : boolean  
var pos := MyRobot._3D.getAttribute("Position", inherited, canInherit)
```

```
// inherited returns if the position can be inherited and canInherit if the  
attribute supports inheritance
```

```
var inherited : boolean  
var pos := MyRobot._3D.getObject("Z").getAttribute("Position", inherited)  
// inherited returns if the position is inherited
```

```
var AttrName : string := "ShowContent"  
var zi := MyFrame._3D.getAttribute(AttrName)
```

See also

[_Methods of User-defined Attributes](#)

[getAttribute \[SimTalk\] - object](#)

[_3D.getMUAnimationPosition \[SimTalk\]](#)

Returns the position on which a *MU* would be animated under certain marginal conditions.

Remarks

You can call `_3D.getMUAnimationPosition` for **length-oriented objects** and for **point-oriented objects**.

getMUAnimationPosition for Length-oriented Objects

Returns the anticipated position of a *MU* on the length-oriented object designated by <Path>.

If you call the method for a **simulation object**, the method evaluates the attribute `_3D.AnimationObject` and, if need be, returns a position on an animated object within the coordinate system of the simulation object.

If you call the method for the **animatable object** itself, the method only considers this object. *Plant Simulation* assumes that the animation will take place on this animatable object and returns the position within the coordinate system of the animatable object.

Type

Method

Syntax

```
<Path>._3D.getMUAnimationPosition(RelPos/AbsPos:real/length[, Lane/  
Direction:integer:=1]) -> length[3]
```

Parameters

You can specify the following parameters:

- The parameter *RelPos/AbsPos* is a value of data type *real* or *length*.
If the value has the data type *real*, it designates the relative position on the object. The relative position can be a value between 0 and 1. 0 designates the start of the animation and 1 the end of the animation, i.e., the value 100 %.
If the value has the data type *length*, it designates the absolute position on the object as the distance starting at the beginning of the object.
- The optional parameter *Lane/Direction* of data type *integer* designates the lane or the direction on the object respectively.
For a *TwoLaneTrack* the value 1 designates **lane A** and the value 2 designates **lane B**.
For a *Converter* the value 1 designates the default direction and the value 2 cross-wise conveying.

For all other objects 1 is the only valid value. If you do not specify the value, *Plant Simulation* automatically uses the value 1.

Default Value of the Parameter

The default value is 1.

Return Value

The return value is an *array* of data type *length* with three values.

Example

```
var p := Converter._3D.getMUAnimationPosition(0.5, 2)
var p := Conveyor._3D.getMUAnimationPosition(1m)
```

getMUAnimationPosition for Point-oriented Objects

Returns the anticipated position of a *MU* on the point-oriented object designated by <Path>.

If you call the method for a **simulation object**, the method evaluates the attribute *_3D.AnimationObject* and, if need be, returns a position on an animated object within the coordinate system of the simulation object.

If you call the method for the **animatable object** itself, the method only considers this object. *Plant Simulation* assumes that the animation will take place on this animatable object and returns the position within the coordinate system of the animatable object.

Type

Method

Syntax

```
<Path>._3D.getMUAnimationPosition([X:integer:=1, Y:integer:=1]) -> length[3]
```

Parameters

You can specify the following parameters:

- The optional parameter **X** of data type *integer* designates the processing place in the X-direction.

You have to enter the parameter for objects with a matrix loading space.

For other objects the parameter is optional. If you do not specify the value, *Plant Simulation* automatically uses the value 1.

For objects, which only have a single processing place, 1 is the only valid value.

Default Value of the Parameter

The default value is 1.

- The optional parameter **Y** of data type *integer* designates the processing place in the Y-direction. You have to enter the parameter for objects with a matrix loading space.

For other objects the parameter is optional. If you do not specify the value, *Plant Simulation* automatically uses the value 1.

For objects, which only have a single processing place, 1 is the only valid value.

Default Value of the Parameter

The default value is 1.

Return Value

The return value is an *array* of data type *length* with three values.

Examples

```
var p := Station._3D.getMUAnimationPosition()
```

```
var p := PlaceBuffer._3D.getMUAnimationPosition(4)
```

```
var p := ParallelStation._3D.getMUAnimationPosition(3,4)
```

```
// Let's assume we have a container on the object Station.  
// Then we can compute the position of the MU on place (1,1)  
// in the current frame as follows:  
var container := Station.Cont  
var animPos   := Station.Cont._3D.getMUAnimationPosition(1,1)  
  
print current._3D.getPositionOfObject(container, animPos)
```

See also

[_3D.MUAnimations.<AnimationPathName>.getMUAnimationPosition \[SimTalk\]](#)

[_3D.AnimationObject \[SimTalk\]](#)

[_3D.getPositionOfObject \[SimTalk\]](#)

[_3D.getMUAnimationRotation \[SimTalk\]](#)

[_3D.Poses.moveToMUAnimationPosition \[SimTalk\]](#)

Video on YouTube

<https://youtu.be/wfVN-mcWNsc?si=X80zVeMID2L2nf-m&t=902>

[_3D.getMUAnimationRotation \[SimTalk\]](#)

Returns the rotation with which a *MU* would be animated under certain marginal conditions.

Remarks

You can call `_3D.getMUAnimationRotation` for **length-oriented objects** and for **point-oriented objects**.

getMUAnimationRotation for Length-oriented Objects

Returns the anticipated rotation of a *MU* on the length-oriented object designated by <Path>.

If you call the method for a **simulation object**, the method evaluates the attribute `_3D.AnimationObject` and, if need be, makes the rotation of the animated object part of the computation.

If you call the method for the **animatable object** itself, the method only considers this object. *Plant Simulation* assumes that the animation will take place on this animatable object and returns the rotation within the coordinate system of the animatable object.

Type

Method

Syntax

```
<Path>._3D.getMUAnimationRotation(RelPos/AbsPos:real/length[, Lane/  
Direction:integer:=1]) -> real[4]
```

Parameters

You can specify the following parameters:

- The parameter *RelPos/AbsPos* is a value of data type *real* or *length*.
If the value has the data type *real*, it designates the rotation at the relative position on the object. The relative position can be a value between 0 and 1. 0 designates the start of the animation and 1 the end of the animation, i.e., the value 100 %.
If the value has the data type *length*, it designates the rotation at the absolute position on the object.
- The optional parameter *Lane/Direction* of data type *integer* designates the lane or the direction on the object respectively.

For a *TwoLaneTrack* the value 1 designates **lane A** and the value 2 designates **lane B**.

For a *Converter* the value 1 designates the default direction and the value 2 cross-wise conveying.

For all other objects 1 is the only valid value. If you do not specify the value, *Plant Simulation* automatically uses the value 1.

Default Value of the Parameter

The default value is 1.

Return Value

The return value is an *array* of data type *real* with four values.

Example

```
var p := Converter._3D.getMUAnimationRotation(0.5, 2)
var p := Conveyor._3D.getMUAnimationRotation(1m)
```

getMUAnimationRotation for Point-oriented Objects

Returns the anticipated rotation of a *MU* on the point-oriented object designated by <Path>.

If you call the method for a **simulation object**, the method evaluates the attribute *_3D.AnimationObject* and, if need be, makes the rotation of the animated object part of the computation.

If you call the method for the **animatable object** itself, the method only considers this object. *Plant Simulation* assumes that the animation will take place on this animatable object and returns the rotation within the coordinate system of the animatable object.

Type

Method

Syntax

```
<Path>._3D.getMUAnimationRotation([X:integer:=1, Y:integer:=1]) -> real[4]
```

Parameters

You can specify the following parameters:

- The optional parameter **X** of data type *integer* designates the processing place in the X-direction.

You have to enter the parameter for objects with a matrix loading space.

For other objects the parameter is optional. If you do not specify the value, *Plant Simulation* automatically uses the value 1.

For objects, which only have a single processing place, 1 is the only valid value.

Default Value of the Parameter

The default value is 1.

- The optional parameter **Y** of data type *integer* designates the processing place in the Y-direction. You have to enter the parameter for objects with a matrix loading space.

For other objects the parameter is optional. If you do not specify the value, *Plant Simulation* automatically uses the value 1.

For objects, which only have a single processing place, 1 is the only valid value.

Default Value of the Parameter

The default value is 1.

Return Value

The return value is an *array* of data type *real* with four values.

Examples

```
var p := Station._3D.getMUAnimationRotation()  
var p := PlaceBuffer._3D.getMUAnimationRotation(4)  
var p := ParallelStation._3D.getMUAnimationRotation(3,4)
```

```
// Let's assume we have a container on the object Station.  
// Then we can compute the rotation of the MU on place (1,1)  
// in the current frame as follows:  
var container := Station.Cont  
var animRot   := Station.Cont._3D.getMUAnimationRotation(1,1)  
print current._3D.getrotationOfObject(container, animPos)
```

See also

[_3D.MUAnimations.<AnimationPathName>.getMUAnimationPosition \[SimTalk\]](#)

[_3D.AnimationObject \[SimTalk\]](#)

[_3D.getPositionOfObject \[SimTalk\]](#)

[_3D.getMUAnimationPosition \[SimTalk\]](#)

[_3D.Poses.moveToMUAnimationPosition \[SimTalk\]](#)

[_3D.getObject \[SimTalk\]](#)

Returns the designated inner object of the object designated by <Path>.

Type

Method

Syntax

```
<Path>._3D.getObject(ObjectName:string)  
<Path>._3D.getObject(ObjectIndex:integer)
```

Parameter

The parameter *ObjectName* of data type *string* designates the name of the object.

Or

The parameter *ObjectIndex* of data type *integer* designates the specified inner object.

Example

```
var o := Machine._3D.getObject("Arm")  
  
// if ObjectName is a valid identifier, you can also type:  
// var o := Machine._3D.Arm
```

Abbreviated Notation if ObjectName is a Valid Identifier

If the parameter *ObjectName* is a valid identifier, you can also type:

```
var o := Machine._3D.Arm
```

See also

[delete \[SimTalk\] - animatable object](#)

[_3D.addObject \[SimTalk\]](#)

[_3D.getGraphic \[SimTalk\]](#)

`_3D.getObject [SimTalk]`

`_3D.getObject [SimTalk]`

[_3D.getPositionOfObject \[SimTalk\]](#)

Returns the position of the designated object in the coordinate system of the object designated by <Path>.

Type

Method

Syntax

```
<Path>._3D.getPositionOfObject(Object:any[, Offset:length[3]]) → length[3]
```

Parameters

You can specify the following parameters:

- The parameter *Object* of data type *any* designates the object whose position you would like to know.
- The optional parameter *Offset* is an *array* of data type *length* with three values. You can use it to convert the designated coordinate within the coordinate system of the passed object into the corresponding coordinate within the coordinate system of the calling object.

Return Value

The return value is an *array* of data type *length* with three values.

It designates the position of the object.

Example

```
// the part Part:1 is located on Transporter:2. The Transporter drives
on .Models.Model.Track ...

Track._3D.getPositionOfObject(.MUs.Part:1) // returns the position of the
Part relative to the object position of the Track
._3D.getPositionOfObject(.MUs.Part:1)      // returns the position of the
Part relative to the Frame,
.MUs.Transporter:2._3D.getPositionOfObject(Station) // returns the
position .Models.Model.Station relative to .Transporter

// You can also specify simulation and 3D simulation objects and animatable
objects.
.MUs.Transporter:2._3D.getPositionOfObject(Station._3D) // same as above
.MUs.Transporter:2._3D.getPositionOfObject(PickAndPlace._3D.getObject(1).get
Object(1).getObject(1))
// returns the position of the ball joint of a four-axis robot relative to
the Transporter
```

See also

[_3D.getRotationOfObject \[SimTalk\]](#)

[_3D.getWorldRotation \[SimTalk\]](#)

[_3D.getWorldCoordinate \[SimTalk\]](#)

[_3D.Position \[SimTalk\]](#)

[_3D.getMUAnimationPosition \[SimTalk\]](#)

Video on YouTube

<https://youtu.be/wfVN-mcWNsc?si=CuqfSgfwl7syzKQc&t=565>

[_3D.getRotationOfObject \[SimTalk\]](#)

Returns the rotation of the designated object in the coordinate system of the object designated by <Path>.

Type

Method

Syntax

```
<Path>._3D.getRotationOfObject(Object:any) → real[4]
```

Parameter

The parameter *Object* of data type *any* designates the object whose rotation you would like to know.

Return Value

The return value is an *array* of data type *real* with four values.

The first value designates the angle, the remaining three values designate the components of the rotation axis.

Example

```
print _3D.getRotationOfObject(Store)
```

See also

[_3D.getPositionOfObject \[SimTalk\]](#)

[_3D.getWorldRotation \[SimTalk\]](#)

[_3D.getWorldCoordinate \[SimTalk\]](#)

[_3D.Rotation \[SimTalk\]](#)

Video on YouTube

<https://youtu.be/wfVN-mcWNsc?si=g56eH65auboJbJYV&t=782>

[_3D.hasAttribute \[SimTalk\]](#)

Returns if the simulation object designated by <Path> has the attribute with the designated name (true) or not (false).

Type

Method

Syntax

```
<Path>._3D.hasAttribute(AttributeName:string) → boolean
```

Parameter

The parameter *AttributeName* of data type *string* designates the *name of the attribute* of which you want to know if the object has it or not.

Return Value

The return value has the data type *boolean*.

true if the object has the attribute that can be read.

false for the parameter *Imp*, *SetupImp*, and *FailImp* if this attribute does not exist for the specified object.

Examples

```
print Store._3D.hasAttribute("StorageAreaMaterialShininess")
```

```
print Station._3D.getObject(1).hasAttribute("AnimationOffset")
```

```
print Conveyor._3D.hasAttribute("Imp")           -- returns false
print Conveyor._3D.hasAttribute("SetupImp")       -- returns false
print Conveyor._3D.hasAttribute("FailImp")        -- returns false
```

See also

[hasAttribute \[SimTalk\] - objects](#)

[_3D.inheritAttribute \[SimTalk\]](#)

Turns inheritance of the attribute designated by *AttributeName* of the object designated by <Path> **on** if it was turned off.

Remarks

If several attributes are inherited together, all attributes will be inherited, when you turn inheritance of a single attribute on.

Type

Method

Syntax

```
<Path>._3D.inheritAttribute(AttributeName:string)
```

Parameter

The parameter *AttributeName* of data type *string* designates the name of the attribute.

Examples

```
Conveyor._3D.inheritAttribute("VisibleGraphicGroups")
```

```
Frame._3D.inheritAttribute("PlanningView")  
// You can call the method for any of the view options.  
// This turns inheritance of all view options on or off.
```

See also

View tab > [Use Planning View, Inherit View Options](#)

[inheritAttribute \[SimTalk\] - object](#)

[_3D.InternalClassType \[SimTalk\]](#)

Returns the built-in English name of the length-oriented object designated by <Path>.

Remarks

The internal class name is not the localized object name that some of the localized versions of *Plant Simulation* show, but the **English** object name that the *Class Library* shows.

Type

Read-only attribute

Syntax

```
<Path>._3D.InternalClassType -> string
```

Return Value

The return value has the data type *string*.

- The return values **AnimatableExtrusionObject** and **SimulationExtrusionObject** designate a length-oriented object in 3D.
- The return values **AnimatableObject** and **SimulationObject** designate a point-oriented object in 3D.

Exceptions are the following objects with the internal class types *Button*, *Chart*, *Checkbox*, *Comment*, *Connector*, *Container*, *Display*, *DropDownList*, *Folder*, *Frame*, *Interface*, *Mixer*, *Part*, *PickAndPlace*, *Store*, *Tank*, *Toolbar*, *Transporter*, *Variable*, and *Worker*.

Example

```
var animObject : any := obj._3D.getObject(1)
if animObject.InternalClassType = "AnimatableExtrusionObject" then
    animObject.BaseHeight := 2
end
```

See also

[InternalClassType \[SimTalk\]](#)

[typeof \[SimTalk\]](#)

[_3D.addLengthOrientation \[SimTalk\]](#)

[_3D.removeLengthOrientation \[SimTalk\]](#)

[_3D.memUsage \[SimTalk\]](#)

Computes how much memory the 3D part of object designated by <Path> uses.

Type

Method

Syntax

```
<Path>._3D.memUsage → integer
```

Return Value

The return value has the data type *integer*.

Example

```
print MyStation._3D.memUsage -- might return 2665, i.e. 2.6 KB
print MyStation.memUsage     -- might return 4975, i.e. 4.9 KB
```

See also

[Show Structure \[class library\]](#)

[memUsage \[SimTalk\]](#)

[_3D.NumObjects \[SimTalk\]](#)

Returns the number of animatable objects that are part of the object designated by <Path>.

Remarks

You can then access these objects with the function `_3D.getObject`.

Type

Read-only attribute

Syntax

```
<Path>._3D.NumObjects → integer
```

Return Value

The return value has the data type *integer*.

Example

```
var obj := .Models.Model.MyStation._3D
for var i := 1 to obj.NumObjects
  print obj.getObject(i).Name
next
```

See also

[_3D.getObject \[SimTalk\]](#)

_3D.openDialog [SimTalk]

Opens the dialog **Edit 3D Properties** of the object designated by <Path>.

Type

Method

Syntax

```
<Path>._3D.openDialog
```

Example

```
Buffer._3D.openDialog
```

See also

[openDialog \[SimTalk\] - all objects](#)

[Edit 3D Properties \[dialog\]](#)

[_3D.openWindow \[SimTalk\]](#)

Opens the object designated by <Path> in a new window in *3D*.

Type

Method

Syntax

```
<Path>._3D.openWindow
```

Example

```
MyFrame._3D.openWindow
```

See also

[Open \[object class\]](#)

[Open in New Window \[simulation object\]](#)

[closeAllWindows \[SimTalk\]](#)

[_3D.removeLengthOrientation \[SimTalk\]](#)

Makes an object out of the *length-oriented object* designated by <Path> which will not be length-oriented.

Remarks

`_3D.removeLengthOrientation` only applies to classes which do not inherit settings from another class.

Type

Method

Syntax

```
<Path>._3D.removeLengthOrientation
```

Example

```
MyStationLengthOriented._3D.removeLengthOrientation
```

See also

[_3D.addLengthOrientation \[SimTalk\]](#)

[_3D.InternalClassType \[SimTalk\]](#)

_3D.Selected [SimTalk]

Selects the object designated by <Path> in 3D (**true**) or deselects it (**false**).

Remarks

_3D.Selected selects all simulation and animatable objects in all windows. The global selection remains active until you close the model or select something else, even if no window is still open.

Type

Attribute

Syntax

```
<Path>._3D.Selected: boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyStation._3D.Selected := true  
MyPickAndPlace._3D.getObject(1).Selected := true
```

See also

Select Control

delete [SimTalk] - animatable object

Deletes the animatable object designated by <PathToObjectToBeDeleted>.

Remarks

Deleting an animatable object can make variables invalid which reference neighboring animatable objects!

It always makes those variables invalid which reference objects which are located within the deleted object.

Type

Method

Syntax

```
<Path>ToObjectToBeDeleted>.delete
```

Example

```
// adds an animatable object to the Station  
var MyAnimatableObject := Station._3D.addObject("MyAnimatableObject")  
// deletes the animatable object from the Station  
MyAnimatableObject.delete  
Station._3D.getObject("MyAnimatableObject").delete
```

See also

[_3D.addObject \[SimTalk\]](#)

[_3D.getObject \[SimTalk\]](#)

Length [SimTalk] - animatable object

Sets the length of the animatable object designated by <Path>.

Remarks

You can set and get this value if the object has a single straight-lined segment.

Otherwise, you can only get this value.

Type

Attribute

Syntax

```
<Path>.Length: length
```

Assignment Value

You can assign a value of data type *length*.

Note

In *SimTalk* 2.0 you can specify the length units m, mm, km, cm, yd, ft, and in.

Type in the unit directly after the values, without a separating blank space, for example 10m or 10.2m.

You can specify the unit for *floating point values* and for *integer values*.

Example

```
MyStation._3D.getObject(1).Length := 2m
```

See also

[_3D.getExtSegments \[SimTalk\]](#)

[_3D.setExtSegments \[SimTalk\]](#)

Name [SimTalk] - animatable object

Sets the name of the animatable object designated by <Path>.

Type

Attribute

Syntax

```
<Path>.Name:string
```

Assignment Value

You can assign a value of data type *string*.

Example

```
PickAndPlace._3D.getObject(1).Name := "MyObject"
var n:string := PickAndPlace._3D.getObject(1).Name
```

See also

[_3D.getObject \[SimTalk\]](#)

[Name \[text box\] - 3D properties](#)

Accessing Transformation Settings

__Accessing Transformation Settings

SimTalk provides the *attributes* and *methods* listed in the table of contents to the left for accessing the transformation of objects in 3D.

To access the attributes in 3D you can type in:

<Path>._3D.NameOfTheAttribute designates the path to the attribute of the 3D object.

To set an attribute, you will type <Path>._3D.NameOfTheAttribute := "MyString"

You can view and access the **attributes and methods** for setting the transformation by clicking **Auto Complete** on the **Edit** ribbon tab of the *Method Editor*.

See also

[Auto Complete](#)

[_3D.CornerPoints \[SimTalk\]](#)

Sets the intermediate anchor points of the *Connector* designated by <Path>.

Type

Attribute

Syntax

```
<Path>._3D.CornerPoints:length[3,*]
```

Assignment Value

You can assign a two-dimensional *array* of data type *length* with sequences of three values.

Examples

```
var cornerPoints := .Models.Model.Connector1._3D.CornerPoints  
cornerPoints[3,1] := 4  
.Models.Model.Connector1._3D.CornerPoints := cornerPoints
```

```
.Models.Model.Connector1._3D.CornerPoints[3,1] := 4
```

See also

[make2DimArray \[SimTalk\]](#)

[_3D.Dimensions \[SimTalk\] - simulation object](#)

Sets the dimensions of the scaled graphic of the simulation object and the non-length-oriented animatable object designated by <Path>.

Remarks

You cannot set any dimensions for an object without graphics.

Note

The following objects do not provide the attribute *_3D.Dimensions*:

- *Part, Container, Transporter, and Worker*
- *Connector, Interface, and Marker*
- *Length-oriented objects*
- *Store*
- *Variable, Comment, Button, DropDownList, Checkbox, and Display*

Type

Attribute

Syntax

```
<Path>._3D.Dimensions:length[3]
```

Assignment Value

You can assign an *array* of data type *length* with three positive values.

Example

```
MyStation._3D.Dimensions := [3m, 2m, 2m]
```

See also

[Edit 3D Properties \[dialog\]](#) > [Transformation](#) > [Settings for the Scale \[transformation\]](#) > [Set Dimensions](#)

[_3D.Dimensions \[SimTalk\] - Store](#)

X [SimTalk] - array

Y [SimTalk] - array

Z [SimTalk] - array

[_3D.getWorldCoordinate \[SimTalk\]](#)

Returns the position of the object designated by <Path> in the topmost *Frame*.

Type

Method

Syntax

```
<Path>._3D.getWorldCoordinate([Offset:length[3]]) → length[3]
```

Parameter

The optional parameter *Offset* is an *array* of data type *length* with three values. You can use it to query the world coordinate of the designated coordinate within the object coordinate system.

Return Value

The return value is an *array* of data type *length* with three values.

Example

```
print Store._3D.getWorldCoordinate(Store.getAnimationPoint(1, 1))
```

See also

[_3D.getWorldRotation \[SimTalk\]](#)

[_3D.getPositionOfObject \[SimTalk\]](#)

[_3D.getRotationOfObject \[SimTalk\]](#)

[_3D.Position \[SimTalk\]](#)

[_3D.getWorldRotation \[SimTalk\]](#)

Returns the rotation of the object designated by <Path> in the topmost *Frame*.

Type

Method

Syntax

```
<Path>._3D.getWorldRotation → real[4]
```

Return Value

The return value is an *array* of data type *real* with four values.

The first value designates the angle, the remaining three values designate the components of the rotation axis.

Example

```
print Store._3D.getWorldRotation // might return [0, 0, 0, -1]
```

See also

[_3D.getPositionOfObject \[SimTalk\]](#)

[_3D.getWorldRotation \[SimTalk\]](#)

[_3D.getWorldCoordinate \[SimTalk\]](#)

[_3D.Rotation \[SimTalk\]](#)

_3D.Mirror [SimTalk]

Sets mirroring of the object designated by <Path> on one or three planes.

Remarks

Mirroring does not apply to existing *length-oriented objects*, *moving objects (MUs)*, and to *Connectors*.

Type

Attribute

Syntax

```
<Path>._3D.Mirror: boolean/boolean[3]
```

Assignment Value

You can specify:

- A single value of data type *boolean* to only mirror the object on the **YZ-Plane**.
- A value of data type *array* containing three values of data type *boolean* to set mirroring around the designated planes. The values of data type *boolean* designate the **YZ-Plane**, the **XZ-Plane**, and the **XY-Plane** in this order.

Examples

```
MyFrame._3D.Mirror := true
```

```
MyFrame._3D.Mirror := [true, false, true]
```

See also

Settings for Mirroring

[_3D.Position \[SimTalk\]](#)

Sets the current position of the object designated by <Path>.

Remarks

_3D.Position does not apply to *MU instances* and to *Connector instances*.

_3D.Position shows the current position.

For *MU instances* this position is computed by the animation.

Type

Attribute

Syntax

```
<Path>._3D.Position:length[3]
```

Assignment Value

You can assign an *array* with three values of data type *length*.

Example

```
MyFrame._3D.Position := [5,3,0]
```

See also

[Settings for the Position \[transformation\]](#)

[_3D.getGraphic \[SimTalk\].Position \[SimTalk\] - graphic](#)

[X \[SimTalk\] - array](#)

[Y \[SimTalk\] - array](#)

[Z \[SimTalk\] - array](#)

[_3D.getWorldCoordinate \[SimTalk\]](#)

_3D.Rotation [SimTalk]

Sets the current rotation of the object designated by <Path>.

Remarks

For an *MU instance*, *_3D.Rotation* will not affect the simulation-relevant rotation of the *MU*. To change the simulation-relevant rotation, use [ConveyingDirection](#) instead.

For an AGV, you can also use the parameter named *Angle* of the method [setWorldPosition](#) to affect the simulation-relevant rotation.

Type

Attribute

Syntax

```
<Path>._3D.Rotation:real/real[4]
```

Assignment Value

You can specify:

- A number which defines the rotation around the negative z-axis at the position at which you inserted the object.
- An *array* of data type *real* with four items, which defines the rotation around the specified axis (values 2 to 4) at the position at which you inserted the object.

If you get the attribute, *3D* returns an *array* with four items.

Examples

```
var a : any := MyStation._3D.Rotation  
// returns an array of 4 values
```

```
MyStation._3D.Rotation := 30  
// rotates the object by 30 degrees
```

```
MyStation._3D.Rotation := [45, 0, 1, 0]  
// rotates the object by 45 degrees around the Y axis
```

See also

[Settings for the Rotation \[transformation\]](#)

[_3D.getGraphic \[SimTalk\].Rotation \[SimTalk\] - graphic](#)

[F3DconcatenateRotations \[SimTalk\]](#)

[_3D.Scale \[SimTalk\]](#)

Sets the current scale of the object designated by <Path>.

Remarks

_3D.Scale does not apply to *Connectors*.

Type

Attribute

Syntax

```
<Path>._3D.Scale: real/real[3]
```

Assignment Value

You can specify:

- A value of data type *real* to achieve uniform scaling.
- An *array* with three values of data type *real* to use different scaling factors for the three axes.

Examples

```
MyStation._3D.Scale := 1           // uniform scaling
```

```
MyStation._3D.Scale := [1, 4, 9]   // different scaling factors for the x-  
axis, y-axis and z-axis
```

See also

[Settings for the Scale \[transformation\]](#)

[Scale \[SimTalk\] - graphic](#)

[X \[SimTalk\] - array](#)

[Y \[SimTalk\] - array](#)

[Z \[SimTalk\] - array](#)

_3D.ScaleAutomatically [SimTalk]

Automatically scales the graphics of the *MU* designated by <MU-Path> (*true*) or not (*false*).

Remarks

Automatically scaling adjusts the size and positioning of the graphic of the *MU* to the measurements and the booking point of the *MU*.

Type

Attribute

Syntax

```
<MU-Path>._3D.ScaleAutomatically:boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
.MUs.Part:1._3D.ScaleAutomatically := true
```

See also

[Scale Automatically](#)

[_3D.exchangeGraphic \[SimTalk\]](#)

[_3D.TransformationMatrix \[SimTalk\]](#)

Sets the entire transformation of the object designated by <Path>.

Remarks

`_3D.TransformationMaxtrix` can either designate a one-dimensional *array* of data type *real* with 16 values or a two-dimensional *array* of data type *real* with four times four values.

`_3D.TransformationMaxtrix` is available for all fully transformable objects (*stations*, *Frames*, *length-oriented objects*, *Interfaces*, and for the classes of the *Connector*, the *MUs*, and the *Worker*).

Note

Assigning the attribute deactivates graphic inheritance.

Type

Attribute

Syntax

```
<Path>._3D.TransformationMatrix → array
```

Return Value

The return value is always an *array* with four times four values.

See also

[Transformation](#)

[addGraphicTransformation \[SimTalk\]](#)

F3DconcatenateRotations [SimTalk]

Concatenates two rotations into one and returns the computed value.

Type

Method

Syntax

```
F3DconcatenateRotations(Rotation1:real[4], Rotation2:real[4]) → real[4]
```

Parameters

You can specify the following parameters:

- The parameter *Rotation1* is an *array* of data type *real* with four values, which sets the first rotation of the graphic. The elements of the *array* denote the rotation angle, and the x-component, the y-component, and the z-component of the rotation axis.
- The parameter *Rotation2* is an *array* of data type *real* with four values, which sets the second rotation of the graphic. The elements of the *array* denote the rotation angle, and the x-component, the y-component, and the z-component of the rotation axis.

Return Value

The return value is an *array* with four values of data type *real*.

It corresponds to the sequence of the input rotations which are executed one after the other in the sequence in which they were passed to the method.

Examples

```
Station._3D.Rotation := F3DconcatenateRotations(Station._3D.Rotation, [45, 0, 1, 0])
```

```
Station._3D.Rotation := F3DconcatenateRotations([90, 0, 1, 0], [45, 0, 1, 0])
```

See also

[_3D.Rotation \[SimTalk\]](#)

[Rotation \[SimTalk\] - graphic](#)

[_3D.NameLabelRotation \[SimTalk\]](#)

Accessing Object Captions

__Accessing Object Captions

SimTalk provides the *attributes* listed in the table of contents to the left for accessing the captions of objects in *3D*.

See also

[Edit 3D Properties \[dialog\] > Captions](#)

[_3D.NameLabelEnabled \[SimTalk\]](#)

Shows (`true`) or hides (`false`) the **Name** and the **Label** of the object designated by `<Path>`.

Remarks

Assigning the attribute deactivates graphic inheritance.

Plant Simulation automatically converts underscores `_` in object names to spaces.

Plant Simulation shows underscores `_` in object labels as underscores.

Type

Attribute

Syntax

```
<Path>._3D.NameLabelEnabled: boolean
```

Assignment Value

You can assign a value of data type *boolean*.

Example

```
MyStation._3D.NameLabelEnabled := true
```

See also

[Edit 3D Properties \[dialog\]](#) > [Captions](#) > [Name/Label Activated](#)

[_3D.NameLabelPosition \[SimTalk\]](#)

Sets the reference point of the **Name** and the **Label** of the object designated by <Path>.

Remarks

Assigning the attribute deactivates graphic inheritance.

Type

Attribute

Syntax

```
<Path>._3D.NameLabelPosition:length/length[3]
```

Assignment Value

You can specify:

- A value of data type *length*.
- A value of data type *array* containing three values of data type *length*.

Example

```
MyStation._3D.NameLabelPosition := [-1, 1, 1]
```

See also

[Edit 3D Properties \[dialog\]](#) > [Captions](#) > [Settings for the Position \[captions\]](#)

[_3D.NameLabelRotation \[SimTalk\]](#)

Sets the rotation of the **Name** and the **Label** of the object designated by <Path>.

Type

Attribute

Syntax

```
<Path>._3D.NameLabelRotation:real/real[4]
```

Assignment Value

You can specify:

- A value of data type *real* that sets the rotation angle in degrees.
- A value of data type *array* containing four values of data type *real*.

Examples

```
MyStation._3D.NameLabelRotation := 45           // degrees
```

```
MyStation._3D.NameLabelRotation := [45, 0, 1, 0] // angle, components of  
the rotation axis
```

See also

[Edit 3D Properties \[dialog\]](#) > [Captions](#)

[_3D.NameLabelScale \[SimTalk\]](#)

Sets the scaling of the **Name** and the **Label** of the object designated by <Path>.

Remarks

Assigning the attribute deactivates graphic inheritance.

Type

Attribute

Syntax

```
<Path>._3D.NameLabelScale:real/real[3]
```

Assignment Value

You can assign different data types:

- You can assign a value of data type *real*, which sets uniform scaling with the factor you specify.
- You can assign a value of data type *array* containing three values of data type *real*, which specify the scaling factor on the **X-axis**, the **Y-axis**, and the **Z-axis**.

Example

```
MyStation._3D.NameLabelScale := [1, 0.7, 0.4]
```

See also

[Edit 3D Properties \[dialog\]](#) > [Captions](#) > [Settings for the Scale \[captions\]](#)

Accessing the Background Color of Frame and Folder

[__Accessing the Background Color of Frame and Folder](#)

SimTalk provides the *attributes* listed in the table of contents to the left for setting the background color of the *Frame* and the *folder* in 3D.

See also

[Edit 3D Properties \[dialog\]](#) > [Background \[tab\]](#) - [Frame](#)

Set the Background Color of the Scene

[_3D.BackgroundBrightness \[SimTalk\]](#)

Sets the brightness of the four corners of background color of the *Frame* or folder designated by <Path>.

Type

Attribute

Syntax

```
<Path>._3D.BackgroundBrightness:real[4]
```

Assignment Value

You can assign a value of data type *array* containing four values of data type *real* between -1 and 1.

They make the color values of the corners of the *Frame* window darker (values less than 0) or brighter (values greater than 0).

- The first value of the *array* designates the top right corner.
- The second value designates the top left corner.
- The third value designates the bottom right corner.
- The fourth value designates the bottom left corner.

Examples

```
MyFrame._3D.BackgroundColor := makeRGBValue(0,0,255)
```

```
MyFrame._3D.BackgroundBrightness := [1, 1, 1, 1]
```

See also

[Edit 3D Properties \[dialog\]](#) > [Background \[tab\]](#) - [Frame](#) > [Corner Brightness](#)

[_3D.BackgroundColor \[SimTalk\]](#)

Sets the background color of the *Frame* or *folder* designated by <Path>.

Type

Attribute

Syntax

```
<Path>._3D.BackgroundColor:integer
```

Assignment Value

You can assign a value of data type *integer*.

If you do not want to define a background color, assign the value -1.

You will usually set the color with the method [makeRGBValue \[SimTalk\]](#).

Example

```
MyFrame._3D.BackgroundColor := makeRGBValue(0,0,255) // bright blue
print MyFrame._3D.BackgroundColor                  // returns 16711680
```

See also

[Edit 3D Properties \[dialog\]](#) > [Background \[tab\]](#) - [Frame](#) > [Base Color](#)

Accessing the Fill Level of the Buffer

[__Accessing the Fill Level of the Buffer](#)

SimTalk provides the *attributes* listed in the table of contents to the left for accessing the fill level of the *Buffer*.

See also

[Show Contents As](#)

[_3D.FillLevelDimensions \[SimTalk\]](#)

Sets the dimensions of the fill level display of the *Buffer* designated by <Path>.

Type

Attribute

Syntax

```
<Path>._3D.FillLevelDimensions:length[2]
```

Assignment Value

You can assign an *array* of data type *length* with two values.

The first value designates the **X-Dimension**, the second designates the **Y-Dimension**.

Example

```
.Models.MyModel.MyBuffer._3D.FillLevelDimensions := [2, 2]
```

See also

[Settings for the Dimensions](#)

[_3D.FillLevelPosition \[SimTalk\]](#)

Sets the position of the fill level display of the *Buffer* designated by <Path>.

Type

Attribute

Syntax

```
<Path>._3D.FillLevelPosition:length[3]
```

Assignment Value

You can assign an *array* with three values of data type *length*

The first value designates the **X-axis**, the second the **Y-axis**, and the third the **Z-axis**.

Example

```
.Models.MyModel.MyBuffer._3D.FillLevelPosition := [0, 0, 0.5]
```

See also

[Settings for the Position \[Buffer\]](#)

[_3D.FillLevelRotation \[SimTalk\]](#)

Sets the rotation of the fill level display of the *Buffer* designated by <Path>.

Type

Attribute

Syntax

```
<Path>._3D.FillLevelRotation:real/real[4]
```

Assignment Value

You can specify:

- A number which defines the rotation around the negative z-axis at the position at which you specified the fill level.
- An *array* with four values. The first value designates the rotation angle. The remaining three values designate the components of the rotation axis at the position at which you specified the fill level.

Example

```
.Models.MyModel.MyBuffer._3D.FillLevelRotation := [0.5, 1, 0, 0]
```

See also

[Settings for the Rotation \[Buffer\]](#)

_3D.ShowContentsAs [SimTalk]

Sets how the *Buffer* designated by <Path> **shows its contents**.

Type

Attribute

Syntax

```
<Path>._3D.ShowContentsAs:string
```

Assignment Value

You can assign a value of data type *string*.

You can specify "MUS", "Fill level", or "Both".

Example

```
.Models.MyModel.MyBuffer._3D.ShowContentsAs := "Both"
```

See also

[Show Contents As](#)

Accessing the Bounding Box

__Accessing the Bounding Box

SimTalk provides the *read-only attributes* listed in the table of contents to the left for measuring the bounding box of the visible graphics of an object.

Note

All data concerning the bounding box relate to the local object coordinate system.

The individual values of the bounding box relate to each other as follows:

_3D.BoundingBoxMin plus *_3D.BoundingBoxMax* divided by 2 results in the *_3D.BoundingBoxCenter*.

_3D.BoundingBoxMax minus *_3D.BoundingBoxMin* results in the *_3D.BoundingBoxSize*.

_3D.BoundingBoxMax has to be equal to or greater than *_3D.BoundingBoxMin*.

[_3D.BoundingBoxCenter \[SimTalk\]](#)

Returns the center of the bounding box of the object designated by <Path>.

Remarks

The bounding box excludes all invisible graphics.

Type

Read-only attribute

Syntax

```
<Path>._3D.BoundingBoxCenter → length[3]
```

Return Value

The return value is an *array* of data type *length* with three values.

The values designate the **X-Position**, the **Y-Position**, and the **Z-Position** of the center point of the bounding box.

Example

```
var a : real[3] := MyStation._3D.BoundingBoxCenter
```

_3D.BoundingBoxMax [SimTalk]

Returns the upper right vertex of the bounding box of the object designated by <Path>.

Remarks

The bounding box excludes all invisible graphics.

The individual values of the bounding box relate to each other as follows:

`_3D.BoundingBoxMin` plus `_3D.BoundingBoxMax` divided by 2 results in the `_3D.BoundingBoxCenter`.

`_3D.BoundingBoxMax` minus `_3D.BoundingBoxMin` results in the `_3D.BoundingBoxSize`.

`_3D.BoundingBoxMax` has to be equal to or greater than `_3D.BoundingBoxMin`.

Type

Read-only attribute

Syntax

```
<Path>._3D.BoundingBoxMax → length[3]
```

Return Value

The return value is an *array* of data type *length* with three values.

The values designate the **X-Position**, the **Y-Position**, and the **Z-Position** of the upper right vertex of the bounding box.

Example

```
var a : length[3] := MyStation._3D.BoundingBoxMax
```

[_3D.BoundingBoxMin \[SimTalk\]](#)

Returns the lower left vertex of the bounding box of the object designated by <Path>.

Remarks

The bounding box excludes all invisible graphics.

The individual values of the bounding box relate to each other as follows:

`_3D.BoundingBoxMin` plus `_3D.BoundingBoxMax` divided by 2 results in the `_3D.BoundingBoxCenter`.

`_3D.BoundingBoxMax` minus `_3D.BoundingBoxMin` results in the `_3D.BoundingBoxSize`.

`_3D.BoundingBoxMax` has to be equal to or greater than `_3D.BoundingBoxMin`.

Type

Read-only attribute

Syntax

```
<Path>._3D.BoundingBoxMin → length[3]
```

Return Value

The return value is an *array* of data type *length* with three values.

The values designate the **X-Position**, the **Y-Position**, and the **Z-Position** of the lower left vertex of the bounding box.

Example

```
var a : length[3] := MyStation._3D.BoundingBoxMin
```

_3D.BoundingBoxSize [SimTalk]

Returns the size of the bounding box of the object designated by <Path>.

Remarks

The bounding box excludes all invisible graphics.

The bounding box of an object encloses its graphics and its animatable objects. In addition, the bounding box of a *Frame* encloses its static simulation objects. A MU instance does not belong to the bounding box of the object that is currently transporting it. The bounding box of a pallet does, for example, not include the parts loaded onto the pallet.

The individual values of the bounding box relate to each other as follows:

$_3D.BoundingBoxMin$ plus $_3D.BoundingBoxMax$ divided by 2 results in the $_3D.BoundingBoxCenter$.

$_3D.BoundingBoxMax$ minus $_3D.BoundingBoxMin$ results in the $_3D.BoundingBoxSize$.

$_3D.BoundingBoxMax$ has to be equal to or greater than $_3D.BoundingBoxMin$.

Type

Read-only attribute

Syntax

```
<Path>._3D.BoundingBoxSize → array
```

Return Value

The return value has the data type *array*.

The return values designate the length (**X-dimension**), the width (**Y-dimension**), and the height (**Z-dimension**) of the bounding box.

Example

```
var a : real[3] := MyStation._3D.BoundingBoxSize
```